PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Construcción de una API REST con Node.js, Express y MySQL

Presentado a:	Alexandra Guevara
Por Aprendiz:	**Luis fernando conejo**
Ficha:	3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 9 del 11 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad 1: Configuración inicial del servidor (Lo que pasa si algo falla)

*¿Qué pasa si no instalas Express? Pues que Node.js queda perdido y no sabe qué carajos es express. Cuando intentes prender el servidor, se te va a apagar de una vez tirando el error de que no encuentra el módulo (Cannot find module 'express'), porque eso no está instalado en las carpetas.

*¿Qué pasa si olvidas ejecutar express.json() y mandas un JSON? El servidor prende normal, pero cuando le mandes datos por POST o PUT desde Postman o Insomnia, el controlador no va a leer nada. El req.body te va a llegar como undefined porque Express ni idea de cómo descifrar ese JSON sin ese middleware.

*¿Qué pasa si el puerto 3000 ya está ocupado? El servidor no va a arrancar y la consola te va a saltar un error rojo (el famoso EADDRINUSE). Para arreglarlo, te toca cerrar la otra app que te esté robando el puerto o cambiarle el número en el app.js.

*¿Qué pasa si escribes mal la ruta de un archivo al importarlo? Node.js se pone a buscar el archivo donde le dijiste, no lo encuentra y te bota el error de que el módulo no existe (Cannot find module...). El servidor se cae de una vez porque no pudo cargar lo que necesitaba.

Actividad 2: Crear funciones de prueba

*Enviar una solicitud POST sin información: El servidor responde normal pero el req.body llega vacío o con propiedades undefined porque no le mandaste ningún JSON en el body.

* Enviar una edad como texto y explicar si el servidor la acepta: Sí la acepta porque JavaScript es flexible y la lee como un string (texto). Sin embargo, si vas a hacer operaciones matemáticas con ella (como sumarle años), te tocará convertirla a número con Number() o parseInt().

* Enviar números en lugar de cadenas al método sumar: Los procesa de una vez sin problema. Y si llegan como texto, con el Number() que le pusimos en el controlador se fuerzan a número para que la suma matemática no falle pegando textos (como pasaría si sumaras "20" + "30" que da "2030" en vez de 50).

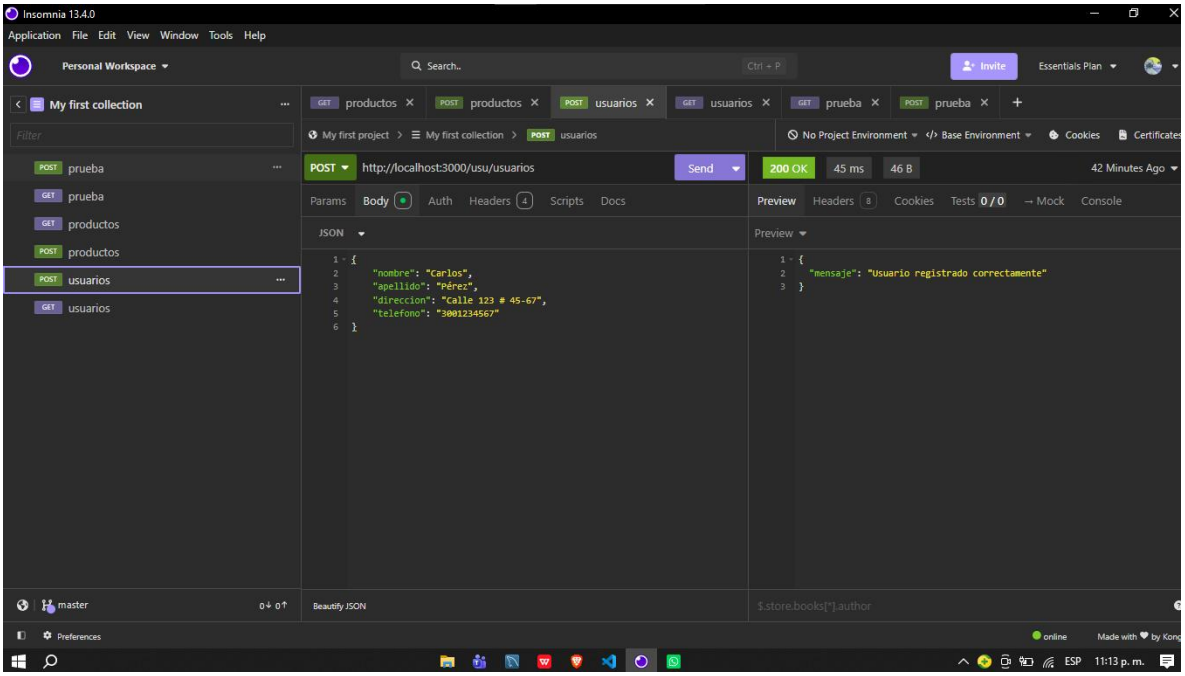
*Omitir uno de los campos requeridos: La API no explota, pero el campo que omitiste llegará con valor undefined en la respuesta (por ejemplo, si mandas solo el número 1, el número 2 quedará como undefined).

* ¿Qué sucede al intentar leer req.body cuando req es undefined? Eso pasaría si por error llamas a una función pasando mal los parámetros (req, res). El servidor tiraría un error de una vez (Cannot read properties of undefined (reading 'body')) porque el objeto de la petición ni siquiera existe en ese contexto.

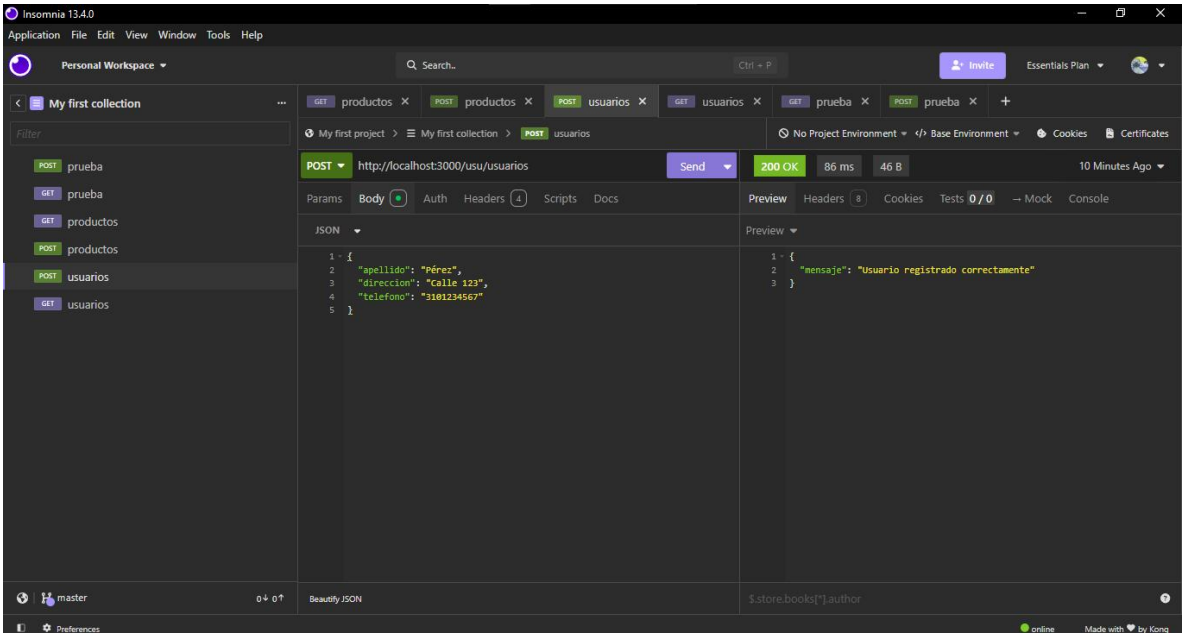
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad 3. Desarrollo del módulo de usuarios

*Registrar un usuario con todos los campos.



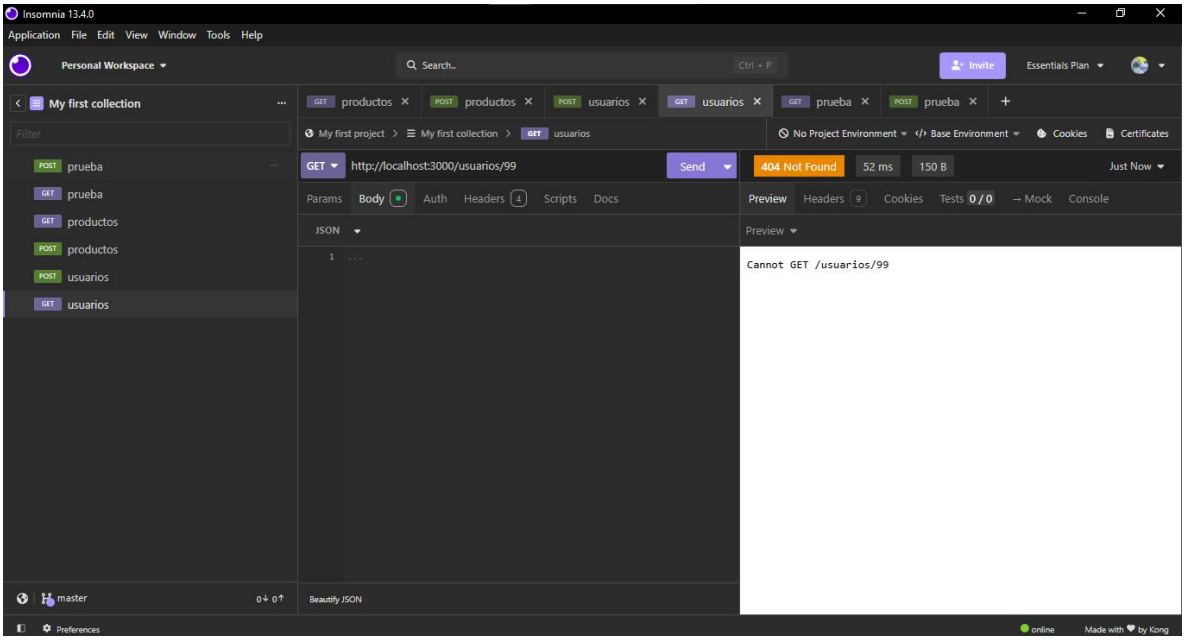
*Intentar registrar un usuario sin nombre o apellido.



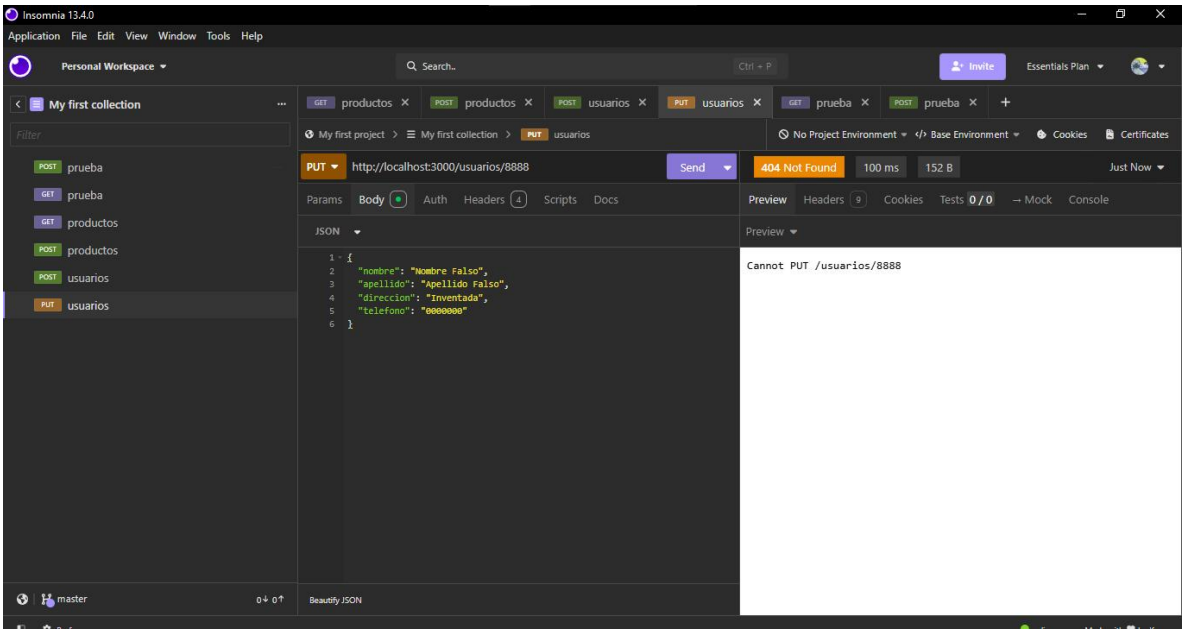
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Intentar registrar un usuario sin nombre o apellido: Le mandamos un POST sin el nombre en el JSON. Aunque Insomnia tiró un verdecito de éxito por cómo está armado el controlador, **en la base de datos no se guardó.** porque la tabla lo frena por ser obligatorio (NOT NULL).

*Consultar un identificador que no existe.

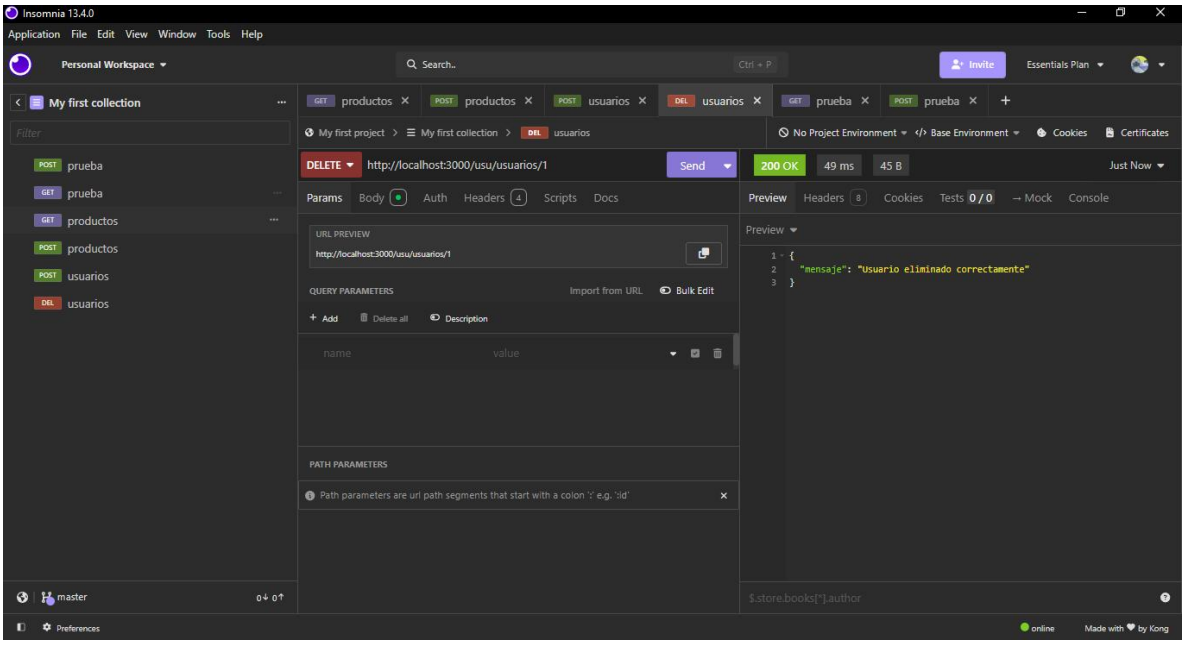


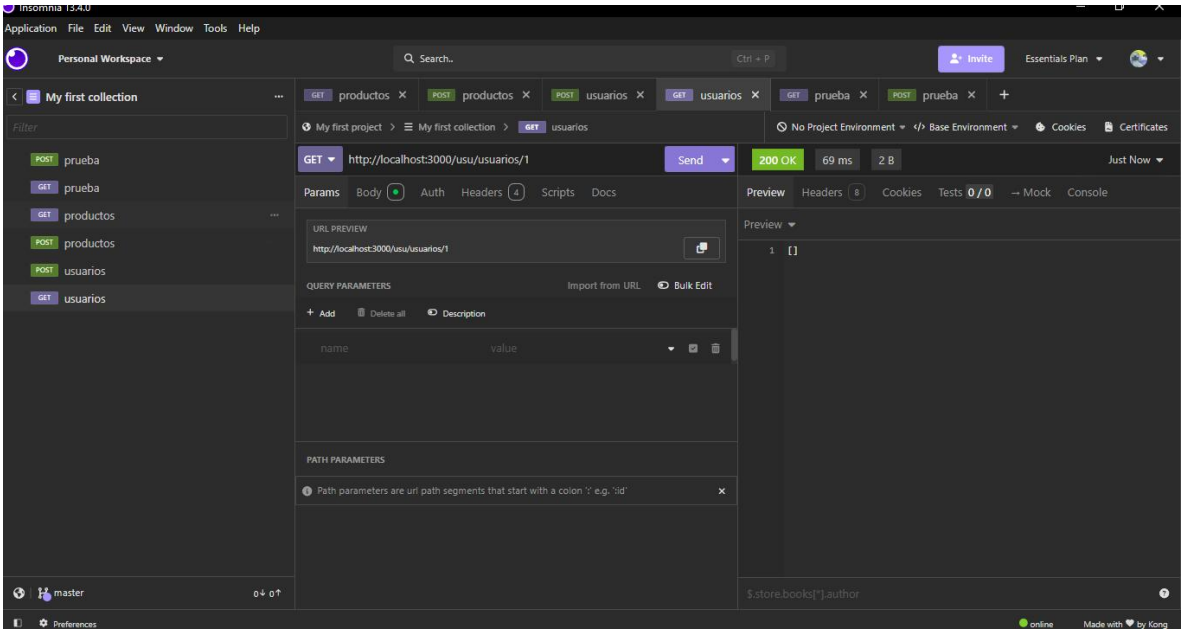
*Actualizar un identificador inexistente.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

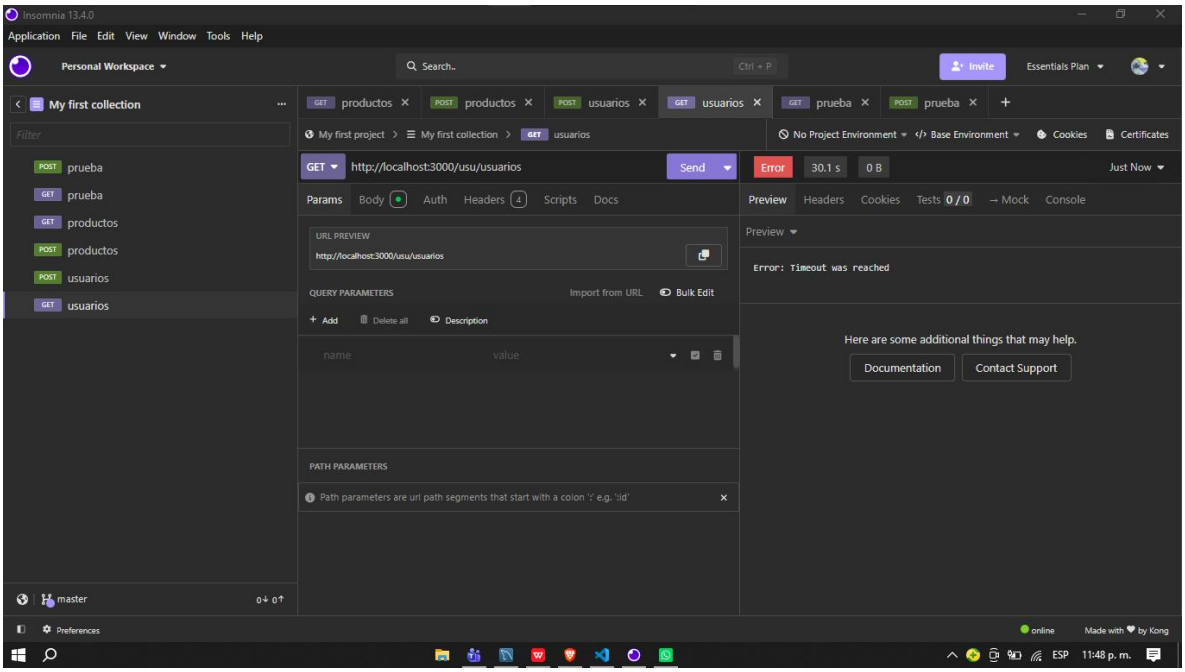
*Eliminar un usuario y volver a consultar el mismo identificador.





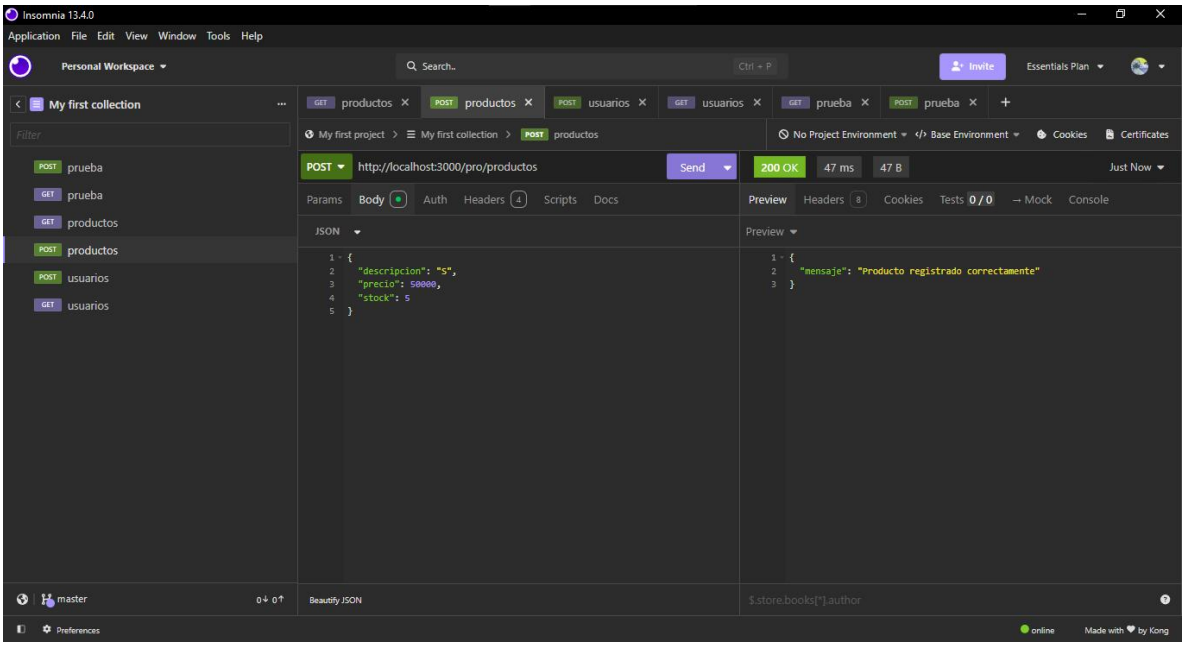
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

*Desconectar MySQL e intentar consultar usuarios.



Actividad 4. Desarrollo del módulo de productos

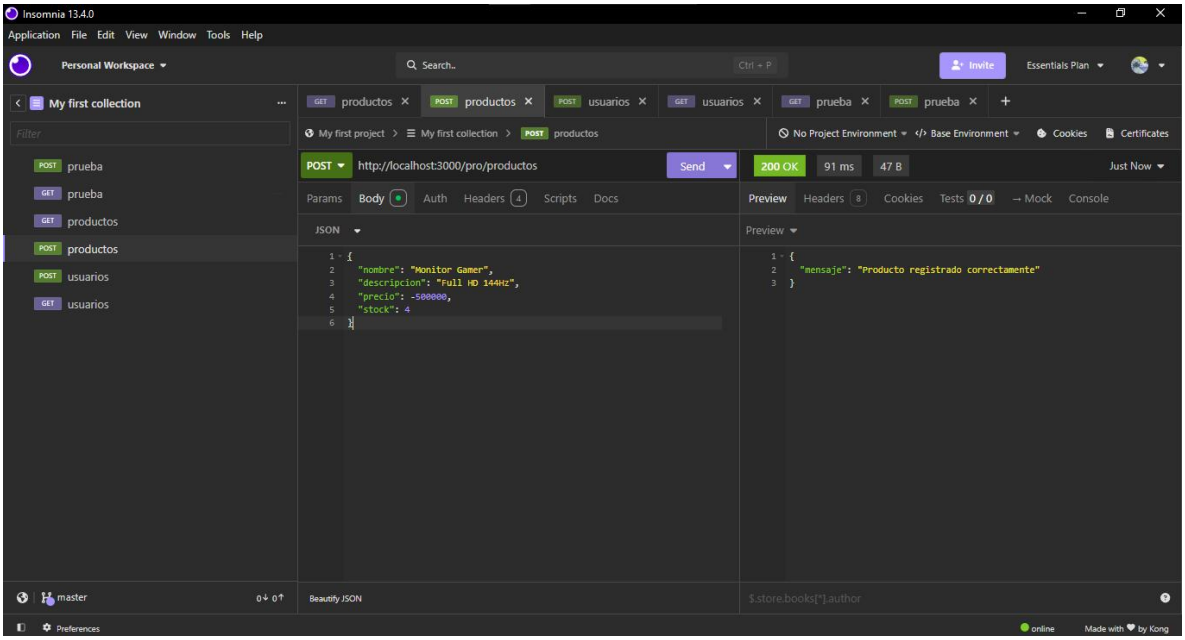
*Intentar registrar un producto sin nombre.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

*Enviar un precio negativo y explicar cómo debería validarse.

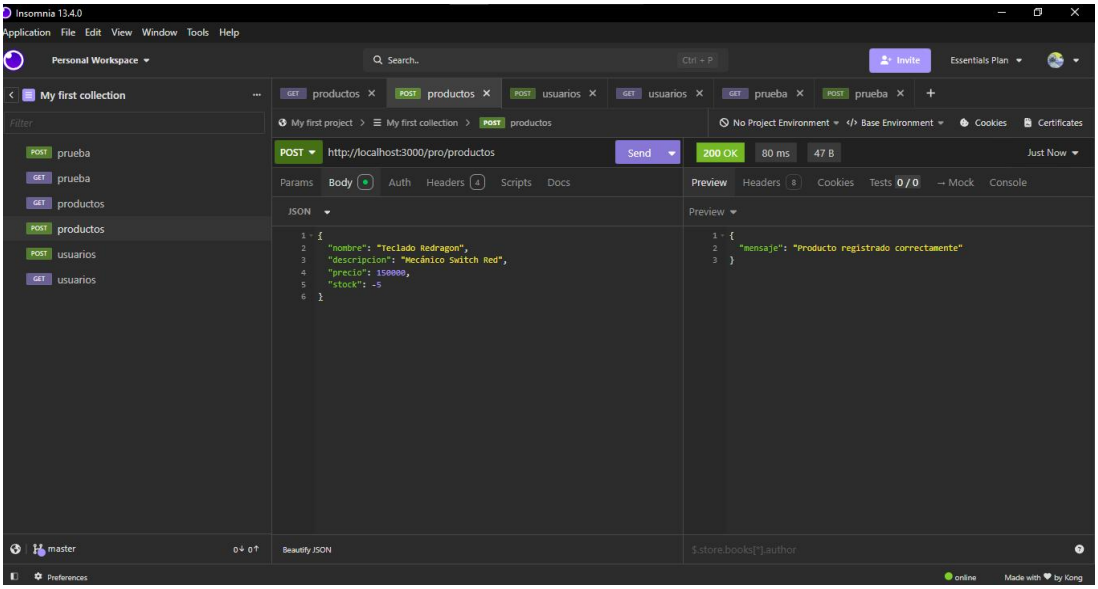
Al enviar un valor menor a cero en el campo del precio, el sistema lo aceptó porque la base de datos no tenía una regla de restricción activa. **Cómo debe validarse:** El controlador de la API debe incluir una validación lógica previa (por ejemplo, comprobar que el precio sea mayor a cero) para rechazar la solicitud de inmediato con un código de error de cliente (400 Bad Request), evitando así que ingresen datos financieros erróneos al inventario.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

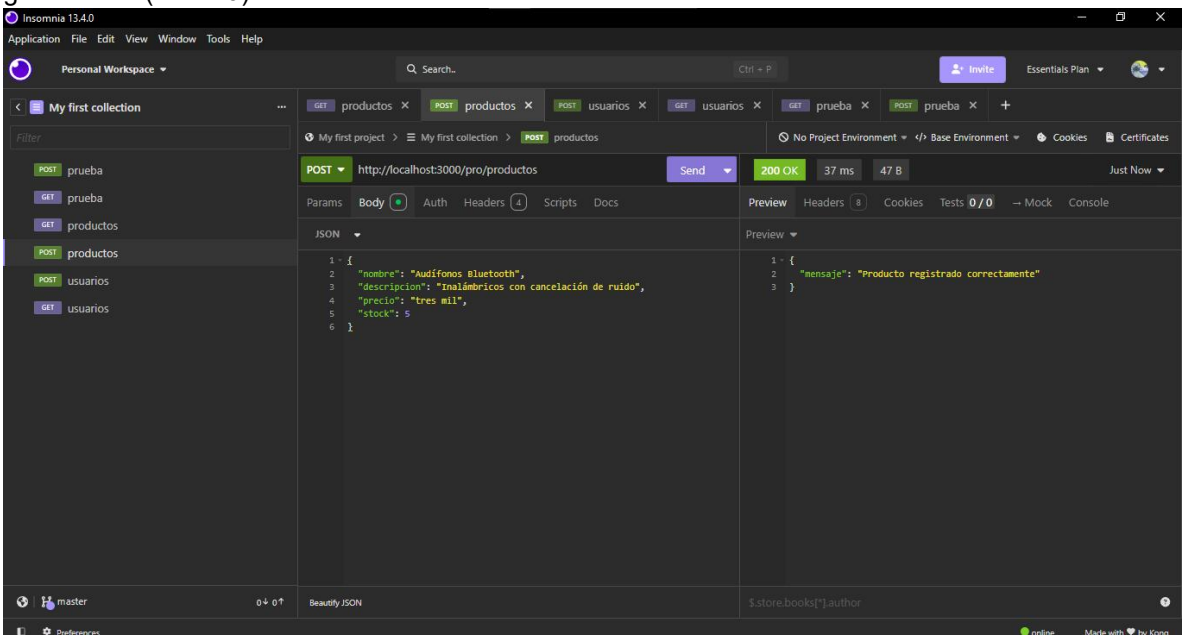
*Enviar un stock negativo y explicar cómo debería validarse.

Al enviar un número menor a cero en la cantidad de inventario, el servidor lo aceptó e ingresó a la base de datos. **Cómo debe validarse:** La API debe validar obligatoriamente por código en el controlador que el stock recibido sea mayor o igual a cero (stock >= 0). Si mandan un número negativo, se debe rechazar la petición con un código 400 Bad Request, ya que en la vida real un producto no puede tener existencias negativas.



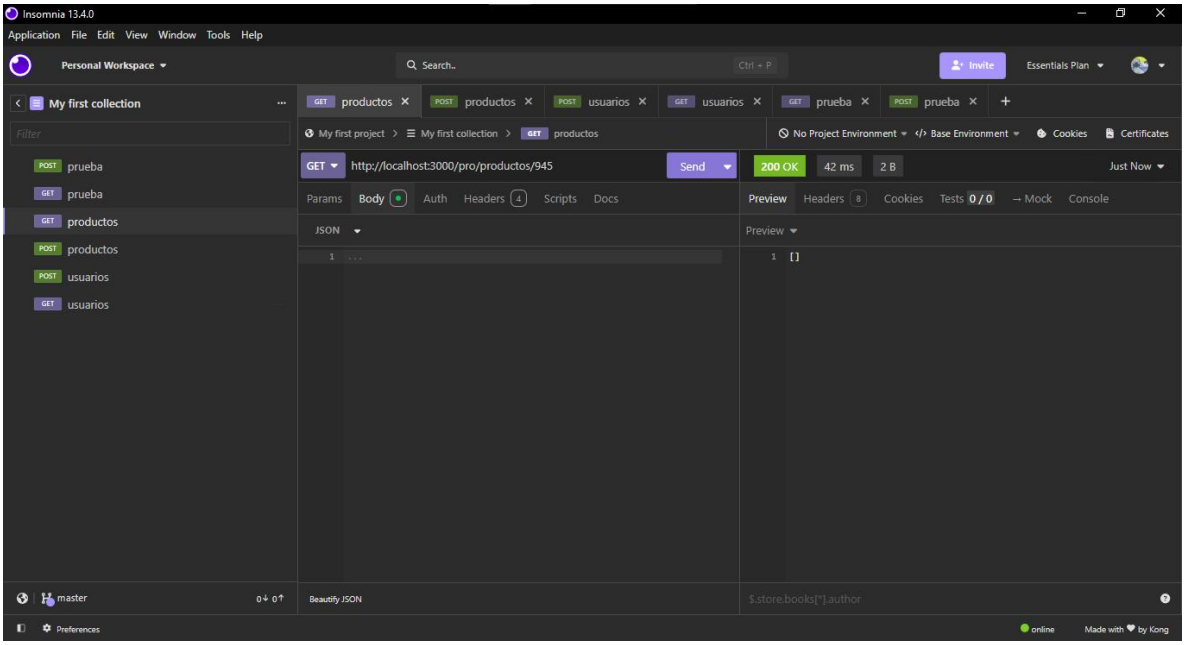
*Enviar el precio como texto no numérico.

Como la API no valida el tipo de dato, la petición pasa, pero la base de datos la rechaza o la guarda mal (como 0)

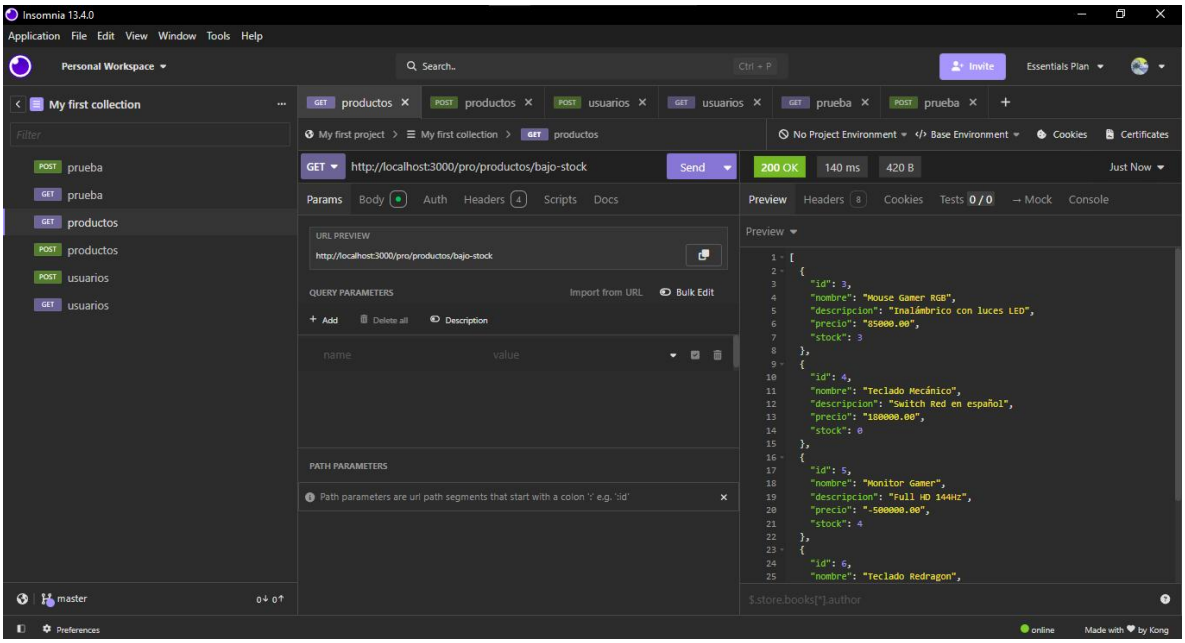


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

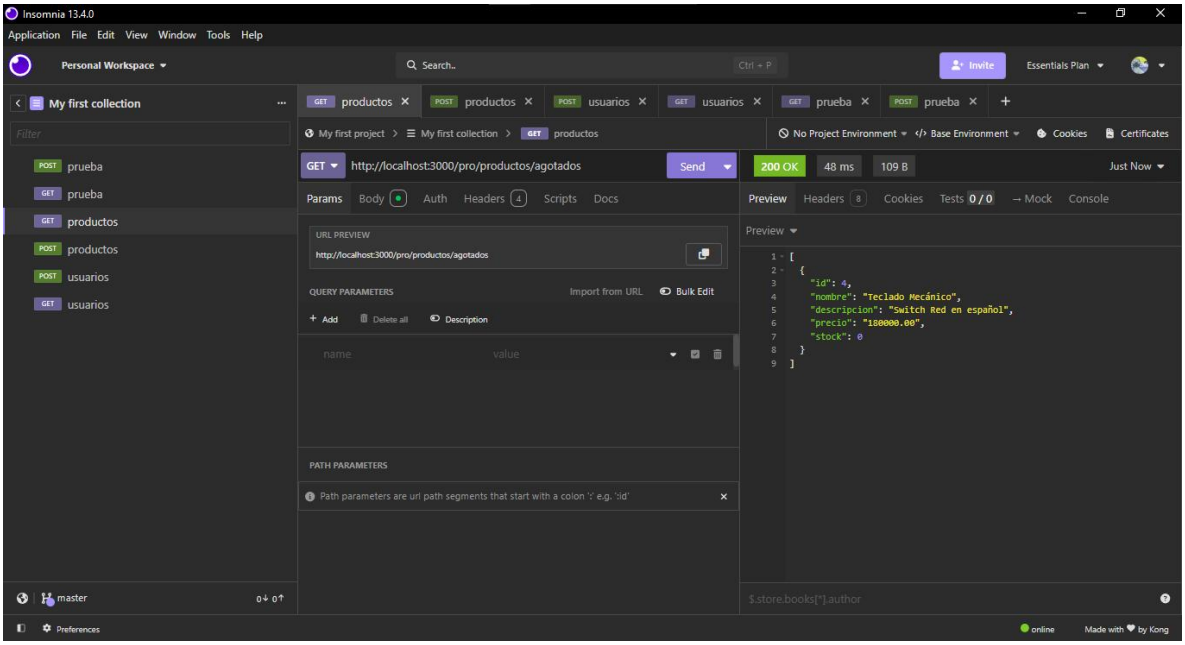
*Buscar un producto inexistente.



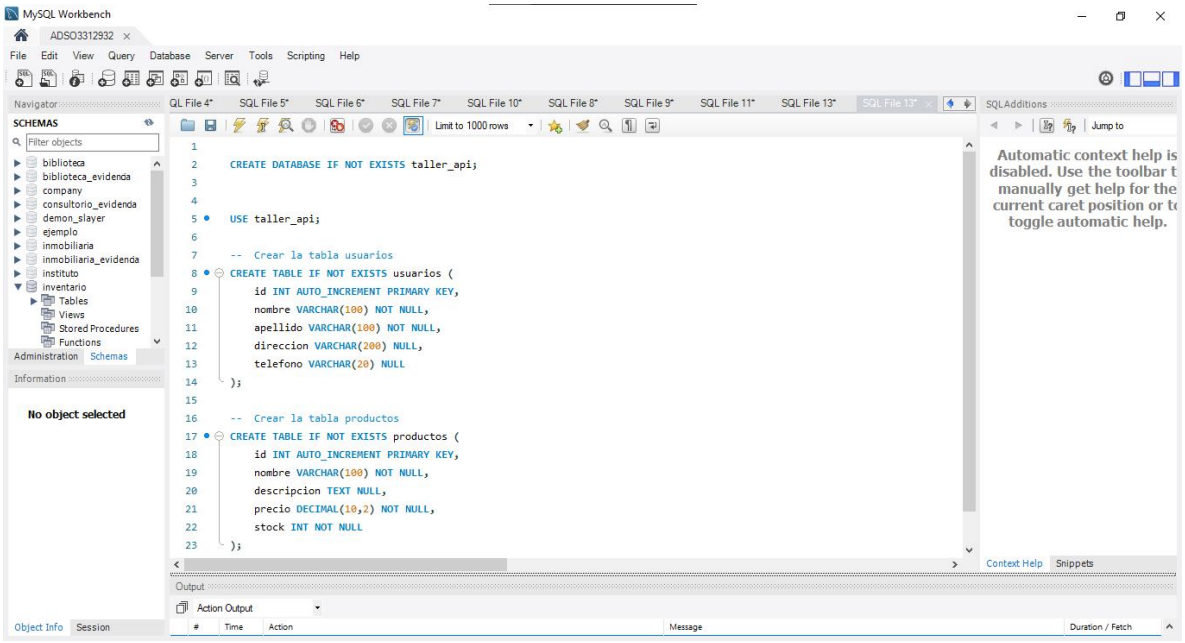
*Consultar productos agotados y productos con bajo inventario.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

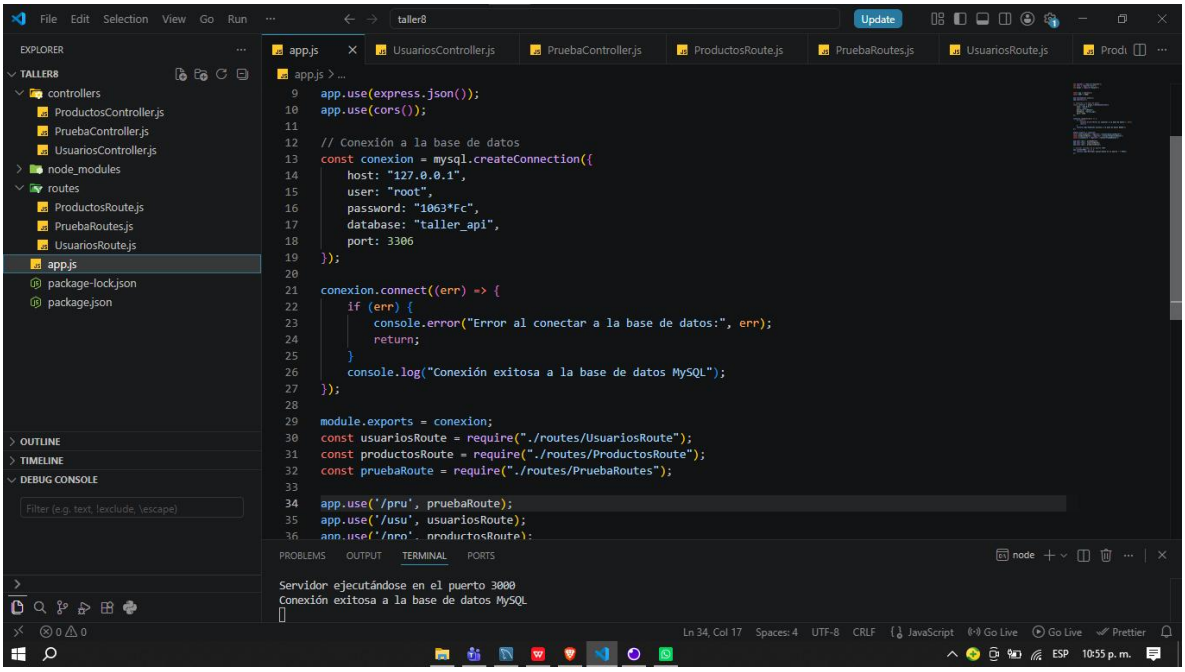


6.Evidencia de la Base de datos



7.Estructura del Proyecto

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Actividad 8: Identificación y Solución de Errores

Error 1

*Error presentado: TypeError: conexion.query is not a function

*Archivo donde ocurrió: ProductosController.js y UsuariosController.js

*Línea aproximada: Línea de la consulta SQL (conexion.query(...))

*Causa identificada: Las rutas se estaban cargando en el app.js antes de que la conexión a la base de datos fuera creada y exportada, haciendo que la variable conexion llegara como undefined a los controladores.

*Solución aplicada: Se reordenó el archivo app.js para que primero se cree la conexión con MySQL, luego se exporte con module.exports = conexion;, y solo después se importen y usen las rutas.

Error 2

*Error presentado: Cannot read properties of undefined (reading 'body')

*Archivo donde ocurrió: En los controladores al recibir un POST

*Línea aproximada: Al intentar desestructurar req.body (ej: const { nombre } = req.body;)

*Causa identificada: Se olvidó agregar el middleware app.use(express.json()); en el app.js, por lo que Express no sabía cómo interpretar el formato JSON que enviaba Postman/Insomnia y dejaba el cuerpo de la petición como undefined.

*Solución aplicada: Se añadió el middleware app.use(express.json()); al principio de la configuración del servidor en el app.js

7. Preguntas de análisis y sustentación

*¿Qué es Node.js y cuál es su función en el proyecto? Es un entorno para correr JavaScript fuera del navegador (usando el motor de Chrome). En el proyecto hace de servidor backend: recibe las peticiones, procesa la lógica y mueve los datos.

*¿Qué es Express y qué ventajas ofrece? Es un framework ligerito para Node.js que te arma servidores y APIs volando. Lo bueno es que es súper fácil de usar, organiza las rutas de una y maneja los middlewares sin enredarte la vida.

*¿Qué es una API REST? Es una forma de comunicar el cliente y el servidor usando las reglas estándar de HTTP (GET, POST, PUT, DELETE) y mandando la info casi siempre en formato JSON.

*¿Qué diferencia existe entre GET, POST, PUT y DELETE?

- GET: Pa' pedir o consultar datos.
- POST: Pa' mandar y registrar datos nuevos.
- PUT: Pa' actualizar o editar un dato que ya existe.
- DELETE: Pa' borrar un registro.

*¿Qué función cumple express.json()? Es un middleware que coge el cuerpo de las peticiones (req.body) que vienen en formato JSON y las convierte en un objeto de JavaScript entendible para el servidor.

*¿Qué es una clase en JavaScript? Es como el molde o la plantilla para crear objetos. Ahí agrupas las características (propiedades) y las acciones (métodos).

*¿Qué es un objeto y cómo se instancia? Es ya el objeto real y tangible creado a partir de esa clase. Se saca usando la palabra new (por ejemplo: const miObjeto = new Clase()).

* ¿Qué función cumple el constructor? Es un método que se dispara solito de inmediato cuando creas un objeto de una clase, y sirve para dejar listas las propiedades principales.

*¿Qué diferencia existe entre un método tradicional y una propiedad con función flecha? El método normal respeta su propio this dependiendo de quién lo llame, mientras que la función flecha hereda el this del entorno que la rodea.

* ¿Para qué se utiliza module.exports? Sirve en Node.js para sacar funciones, clases o rutas de un archivo y permitir que otros archivos las puedan importar.

*¿Qué función cumple require()? Es la orden nativa de Node.js para importar o jalar módulos, librerías o controladores de otros archivos y usarlos donde los necesites.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

* ¿Qué responsabilidad tiene un Controller? Manejar toda la lógica de negocio: recibe la petición del cliente, habla con la base de datos y manda de vuelta la respuesta HTTP correcta.

*¿Qué responsabilidad tiene Routes? Poner las direcciones (endpoints) de la URL y conectar cada método HTTP con la función que le toca en el controlador.

*¿Por qué la conexión MySQL se comparte desde app.js? Para centralizar todo en un solo lugar y no abrir conexiones locas por todos lados, reutilizando la misma base de datos en toda la app.

*¿Qué función cumplen los signos ? en las consultas SQL parametrizadas? Funcionan como escudo protector contra ataques de inyección SQL, separando los datos que escribe el usuario de la estructura del comando SQL.

*¿Por qué es importante controlar los errores en una API? Para evitar que el servidor se estrelle (se caiga), y para devolverle al usuario un mensaje claro y un código de error decente en lugar de dejar la pantalla en blanco.